

DOSSIER D'ARCHITECTURE LOGICIELLE

Refonte IAM & RBAC : Migration vers Keycloak (V2)

Projet : IncidentFlow | Module : Sécurité & Gestion des Accès

1. Objectif de la Migration

Dans la première itération du projet (V1), la gestion des identités, le stockage des habilitations, ainsi que le hachage des mots de passe étaient pris en charge directement par le code de l'application métier. Dans le cadre d'une montée en version vers les standards d'architecture d'entreprise (V2), la responsabilité de la gestion des identités et des accès (IAM) a été externalisée vers le fournisseur d'identités **Keycloak**.

Cette approche permet de garantir une séparation stricte des préoccupations, de renforcer la sécurité des données d'authentification et de préparer le système à une potentielle fédération d'identités (Single Sign-On).

2. Architecture Multi-Tenant (Concepts de Realm)

L'architecture de Keycloak repose sur le concept d'isolation logique appelé "Realm" (Royaume). Ce principe permet d'héberger plusieurs environnements distincts sur un même serveur.

Le Realm "master"

Environnement réservé exclusivement à l'administration de l'infrastructure Keycloak. Il est isolé de la logique applicative métier et ne contient aucun compte utilisateur lié au système IncidentFlow.

Le Realm "incidentflow-realm"

Espace locataire (tenant) dédié au système. Il centralise les comptes utilisateurs de l'entreprise, les politiques de sécurité (complexité des mots de passe) et la topologie des rôles applicatifs.

3. Modélisation RBAC Avancée (Rôles Composites)

La gestion des droits d'accès s'appuie sur une structure RBAC (Role-Based Access Control) fine, exploitant la fonctionnalité des rôles composites de Keycloak afin d'éviter la gestion en dur des permissions dans le code source.

Realm Roles (Rôles Métiers Globaux)

- └─ Administrateur
- └─ Responsable
- └─ Opérateur

Client Roles (Permissions Techniques du Frontend)

- └─ incidentflow-frontend: INCIDENT_CREATE
- └─ incidentflow-frontend: PAGE_DASHBOARD
- └─ incidentflow-frontend: INCIDENT_DELETE

La structuration métier repose sur l'encapsulation : le rôle métier "Opérateur" est configuré dans Keycloak comme un rôle composite héritant automatiquement des permissions techniques PAGE_DASHBOARD et INCIDENT_CREATE. Le client (React) et le serveur (Spring Boot) analysent le jeton JWT (JSON Web Token) généré par Keycloak pour appliquer les contrôles d'accès dynamiquement, supprimant ainsi le besoin de mise à jour du code lors de l'évolution de la matrice des droits.

4. Impacts Techniques et Restructuration

La transition de la V1 à la V2 a induit les modifications structurelles suivantes sur les différentes couches de l'application :

Base de Données (PostgreSQL)	Obsolescence des tables locales relatives à la sécurité (users, roles, permissions). Les entités d'authentification sont désormais externalisées dans le schéma interne de Keycloak.
Frontend (React)	Suppression du code lié aux interfaces CRUD de gestion des utilisateurs. Remplacement des vues <code>UserManagementView.jsx</code> et <code>RoleManagementView.jsx</code> par des points de redirection (portails IAM) vers la console d'administration Keycloak.
Backend (Spring Boot 3)	Remplacement du protocole d'authentification local par l'implémentation du standard OAuth2 Resource Server . Le backend délègue la validation de l'identité en vérifiant la signature cryptographique asymétrique du jeton JWT émis par Keycloak.

**Infrastructure
(Docker)**

Ajout du service `incidentflow-keycloak-v2` au fichier `docker-compose.yml`. Intégration d'un processus d'auto-provisionnement via le fichier `realm-export.json` pour l'initialisation automatisée du royaume et des rôles.